

教学管理服务 - 学生信息查询报告

一、实验目的

本实验通过编写Python脚本，调用教学管理服务API，实现学生信息的查询功能，并将查询结果输出为PDF报告。

二、实验环境

- ☐ 编程语言：Python 3.x
- ☐ 依赖库：requests, reportlab
- ☐ API地址：<https://pysummer.pkuai.cc/api/v1>

三、Python代码

```
1 import requests
2 import json
3 import sys
4 import os
5 import argparse
6 from typing import Optional, Dict, Any
7 from reportlab.lib.pagesizes import letter
8 from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
9 from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
10 from reportlab.lib import colors
11 from reportlab.pdfbase import pdfmetrics
12 from reportlab.pdfbase.ttfonts import TTFont
13
14 DEFAULT_BASE_URL = "https://pysummer.pkuai.cc/api/v1"
15 MAX_RETRIES = 2
16 TIMEOUT = 15
17
18 try:
19     pdfmetrics.registerFont(TTFont('SimHei', 'simhei.ttf'))
20 except:
21     pass
22
23
24 def create_session(verify_ssl: bool = True) -> requests.Session:
25     session = requests.Session()
```

```

26     session.headers.update({
27         "User-Agent": "StudentInfoQuery/1.0",
28         "Accept": "application/json",
29         "Content-Type": "application/json; charset=utf-8",
30     })
31     session.verify = verify_ssl
32     return session
33
34
35 def validate_input(student_id: str, secret: str, base_url: str) -> bool:
36     if not student_id or not student_id.strip():
37         print("[ ]      ")
38         return False
39     if not secret or not secret.strip():
40         print("[ ]      ")
41         return False
42     if not base_url or not base_url.strip():
43         print("[ ] API      ")
44         return False
45     return True
46
47
48 def api_login(session: requests.Session, base_url: str, student_id: str, secret: str) ->
Optional[str]:
49     url = f"{base_url}/auth/login"
50     payload = {"identifier": student_id.strip(), "secret": secret.strip()}
51
52     for attempt in range(MAX_RETRIES):
53         try:
54             resp = session.post(url, json=payload, timeout=TIMEOUT)
55
56             if resp.status_code == 401:
57                 try:
58                     err_data = resp.json()
59                     msg = err_data.get("error", {}).get("message", " ")
60                 except:
61                     msg = " "
62                 print(f"[ ] {msg}")
63                 return None
64
65             resp.raise_for_status()
66
67             try:
68                 data = resp.json()
69             except json.JSONDecodeError:
70                 print(f"[ ]      JSON : {resp.text[:200]}")
71                 return None
72
73             if "access_token" not in data:

```

```

74         print("[ ]      access_token  ")
75         return None
76
77     return data["access_token"]
78
79     except requests.exceptions.ConnectionError:
80         print(f"[ ]      ( {attempt + 1}/{MAX_RETRIES})")
81         if attempt < MAX_RETRIES - 1:
82             print("    ...")
83     except requests.exceptions.Timeout:
84         print(f"[ ]      ( {attempt + 1}/{MAX_RETRIES})")
85         if attempt < MAX_RETRIES - 1:
86             print("    ...")
87     except requests.exceptions.SSLError:
88         print("[ ] SSL      --no-ssl-verify  ")
89         return None
90     except requests.exceptions.HTTPError as e:
91         print(f"[ ] HTTP : {e}")
92         try:
93             err_data = resp.json()
94             if "error" in err_data:
95                 err_code = err_data["error"].get("code", "")
96                 err_msg = err_data["error"].get("message", "")
97                 print(f"[ ] [{err_code}] {err_msg}")
98         except:
99             pass
100         return None
101     except Exception as e:
102         print(f"[ ]      : {str(e)}")
103         return None
104
105     print("[ ]      ")
106     return None
107
108
109 def get_student_info(session: requests.Session, base_url: str, token: str) ->
Optional[Dict[str, Any]]:
110     url = f"{base_url}/students/me"
111     session.headers["Authorization"] = f"Bearer {token}"
112
113     for attempt in range(MAX_RETRIES):
114         try:
115             resp = session.get(url, timeout=TIMEOUT)
116
117             if resp.status_code == 401:
118                 try:
119                     err_data = resp.json()
120                     msg = err_data.get("error", {}).get("message", "Token      ")
121                 except:

```

```

122         msg = "Token      "
123         print(f"[ ] {msg}")
124         return None
125
126     resp.raise_for_status()
127
128     try:
129         data = resp.json()
130     except json.JSONDecodeError:
131         print(f"[ ]      JSON : {resp.text[:200]}")
132         return None
133
134     return data
135
136 except requests.exceptions.ConnectionError:
137     print(f"[ ]      ( {attempt + 1}/{MAX_RETRIES})")
138     if attempt < MAX_RETRIES - 1:
139         print("    ...")
140 except requests.exceptions.Timeout:
141     print(f"[ ]      ( {attempt + 1}/{MAX_RETRIES})")
142     if attempt < MAX_RETRIES - 1:
143         print("    ...")
144 except requests.exceptions.SSLError:
145     print("[ ] SSL      ")
146     return None
147 except requests.exceptions.HTTPError as e:
148     print(f"[ ] HTTP : {e}")
149     try:
150         err_data = resp.json()
151         if "error" in err_data:
152             err_code = err_data["error"].get("code", "")
153             err_msg = err_data["error"].get("message", "")
154             print(f"[ ] [{err_code}] {err_msg}")
155     except:
156         pass
157     return None
158 except Exception as e:
159     print(f"[ ]      : {str(e)}")
160     return None
161
162 print(f"[ ]      ")
163 return None
164
165
166 def print_info(info: Dict[str, Any]):
167     print("\n" + "=" * 35)
168     print("      ")
169     print("-" * 35)
170

```

```

171     student_id = info.get("student_id", "N/A")
172     print(f"      : {student_id}")
173
174     name = info.get("name", "N/A")
175     print(f"      : {name}")
176
177     extra = info.get("extra", {})
178     if extra:
179         try:
180             extra_str = json.dumps(extra, ensure_ascii=False, indent=2)
181             print(f"      :\n{extra_str}")
182         except:
183             print(f"      : {extra}")
184     else:
185         print("      : ")
186
187     created_at = info.get("created_at", "N/A")
188     print(f"      : {created_at}")
189
190     updated_at = info.get("updated_at", "N/A")
191     print(f"      : {updated_at}")
192
193     print("=" * 35)
194
195
196 def generate_markdown(info: Dict[str, Any]) -> str:
197     student_id = info.get("student_id", "N/A")
198     name = info.get("name", "N/A")
199     extra = info.get("extra", {})
200     created_at = info.get("created_at", "N/A")
201     updated_at = info.get("updated_at", "N/A")
202
203     md_content = f"""#
204
205 ##
206
207 |      |      |
208 |-----|-----|
209 |      | {student_id} |
210 |      | {name} |
211 |      | {created_at} |
212 |      | {updated_at} |
213
214 ##
215
216 """
217
218     if extra:
219         try:

```

```

220         extra_str = json.dumps(extra, ensure_ascii=False, indent=2)
221         md_content += f"```json\n{extra_str}\n```"
222     except:
223         md_content += f"{extra}"
224 else:
225     md_content += " "
226
227     return md_content
228
229
230 def save_markdown(info: Dict[str, Any], filename: str = None) -> str:
231     if not filename:
232         student_id = info.get("student_id", "student")
233         filename = f"_{student_id}.md"
234
235     md_content = generate_markdown(info)
236
237     with open(filename, 'w', encoding='utf-8') as f:
238         f.write(md_content)
239
240     return filename
241
242
243 def save_pdf(info: Dict[str, Any], filename: str = None) -> str:
244     if not filename:
245         student_id = info.get("student_id", "student")
246         filename = f"_{student_id}.pdf"
247
248     doc = SimpleDocTemplate(filename, pagesize=letter)
249     styles = getSampleStyleSheet()
250
251     try:
252         title_style = ParagraphStyle(
253             'TitleStyle',
254             parent=styles['Title'],
255             fontName='SimHei',
256             fontSize=18,
257             alignment=1,
258             spaceAfter=20
259         )
260         normal_style = ParagraphStyle(
261             'NormalStyle',
262             parent=styles['Normal'],
263             fontName='SimHei',
264             fontSize=12,
265             spaceAfter=10
266         )
267         header_style = ParagraphStyle(
268             'HeaderStyle',

```

```

269         parent=styles['Heading2'],
270         fontName='SimHei',
271         fontSize=14,
272         spaceAfter=15
273     )
274 except:
275     title_style = styles['Title']
276     normal_style = styles['Normal']
277     header_style = styles['Heading2']
278
279 elements = []
280
281 elements.append(Paragraph("      ", title_style))
282 elements.append(Spacer(1, 12))
283
284 elements.append(Paragraph("      ", header_style))
285
286 student_id = info.get("student_id", "N/A")
287 name = info.get("name", "N/A")
288 created_at = info.get("created_at", "N/A")
289 updated_at = info.get("updated_at", "N/A")
290
291 data = [
292     [" ", " "],
293     [" ", student_id],
294     [" ", name],
295     [" ", created_at],
296     [" ", updated_at]
297 ]
298
299 table = Table(data, colWidths=[100, 400])
300 table.setStyle(TableStyle([
301     ('BACKGROUND', (0, 0), (-1, 0), colors.grey),
302     ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
303     ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
304     ('FONTNAME', (0, 0), (-1, -1), 'SimHei'),
305     ('FONTSIZE', (0, 0), (-1, -1), 12),
306     ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
307     ('BACKGROUND', (0, 1), (-1, -1), colors.beige),
308     ('GRID', (0, 0), (-1, -1), 1, colors.black),
309 ]))
310
311 elements.append(table)
312 elements.append(Spacer(1, 12))
313
314 elements.append(Paragraph("      ", header_style))
315
316 extra = info.get("extra", {})
317 if extra:

```

```

318         try:
319             extra_str = json.dumps(extra, ensure_ascii=False, indent=2)
320             elements.append(Paragraph(f"{extra_str}", normal_style))
321         except:
322             elements.append(Paragraph(str(extra), normal_style))
323     else:
324         elements.append(Paragraph(" ", normal_style))
325
326     doc.build(elements)
327
328     return filename
329
330
331 def get_input(prompt: str, allow_empty: bool = False) -> str:
332     while True:
333         try:
334             value = input(prompt).strip()
335             if value or allow_empty:
336                 return value
337             print("[ ]")
338         except KeyboardInterrupt:
339             print("\n[ ]")
340             sys.exit(0)
341         except EOFError:
342             print("\n[ ]")
343             sys.exit(0)
344
345
346 def main():
347     parser = argparse.ArgumentParser(description=" - ")
348     parser.add_argument("-s", "--student-id", help=" ")
349     parser.add_argument("-k", "--secret", help=" ")
350     parser.add_argument("--export-md", action="store_true", help=" Markdown ")
351     parser.add_argument("--export-pdf", action="store_true", help=" PDF ")
352     parser.add_argument("--export-all", action="store_true", help=" Markdown PDF ")
353
354     args = parser.parse_args()
355
356     if args.student_id and args.secret:
357         student_id = args.student_id.strip()
358         secret = args.secret.strip()
359     else:
360         print("=" * 40)
361         print(" - ")
362         print("=" * 40)
363         print(" ")
364         print(" Ctrl+C ")
365         print("-" * 40)
366

```



```

367     student_id = get_input("      : ")
368     secret = get_input("      : ")
369
370     print("\n      ...")
371
372     session = create_session(verify_ssl=True)
373
374     print("      ...")
375     token = api_login(session, DEFAULT_BASE_URL, student_id, secret)
376
377     if not token:
378         print("\n[ ]      ")
379         sys.exit(1)
380
381     print("      ")
382     print("      ...")
383
384     info = get_student_info(session, DEFAULT_BASE_URL, token)
385
386     if not info:
387         print("[ ]      ")
388         sys.exit(1)
389
390     print_info(info)
391
392     export_md = args.export_md or args.export_all
393     export_pdf = args.export_pdf or args.export_all
394
395     if not args.student_id and not args.secret:
396         print("\n" + "-" * 40)
397         print("      ")
398         print("1.      Markdown      ")
399         print("2.      PDF      ")
400         print("3.      Markdown      PDF")
401         print("4.      ")
402
403         choice = get_input("      (1-4): ")
404
405         while choice not in ["1", "2", "3", "4"]:
406             print("[ ]      (1-4)")
407             choice = get_input("      (1-4): ")
408
409         export_md = choice in ["1", "3"]
410         export_pdf = choice in ["2", "3"]
411
412     if export_md:
413         try:
414             md_file = save_markdown(info)
415             print(f"[ ] Markdown      : {os.path.abspath(md_file)}")

```

```
416         except Exception as e:
417             print(f"[ ] Markdown      : {str(e)}")
418
419     if export_pdf:
420         try:
421             pdf_file = save_pdf(info)
422             print(f"[ ] PDF          : {os.path.abspath(pdf_file)}")
423         except Exception as e:
424             print(f"[ ] PDF          : {str(e)}")
425
426     print("\n      ")
427
428
429 if __name__ == "__main__":
430     main()
```

四、实验结果

实验 1

输入参数：

- ☐ 学号：2600926009
- ☐ 密钥：p22GhAl0xg62tDRLphehWN9kIg4Akf0s

****查询结果：** 成功**

学生信息

项目	内容
-----	-----
学号	2600926009
姓名	陈斌
创建时间	2026-07-19T22:03:03+08:00
更新时间	2026-07-19T22:18:35+08:00

扩展信息

无

实验 2

输入参数：

- ☐ 学号：2600940418
- ☐ 密钥：vSJme4mA5la6Vn8Ao58UdNEgRsEkgo-m

查询结果： 成功

学生信息

项目	内容
-----	-----
学号	2600940418
姓名	潘静怡
创建时间	2026-07-19T17:10:14+08:00
更新时间	2026-07-19T17:10:14+08:00

扩展信息

无

报告生成时间： 2026-07-23